Rasoi (रसोई) AI

Ingredient Detection & Recipe Generation Pipeline

Food Computing Capstone Project

Vinayak Goswamy

Abstract

This report presents **Rasoi AI**, an extended end-to-end pipeline that accepts a photograph of kitchen ingredients and automatically generates a complete recipe. The system has been substantially extended since its initial implementation. The pipeline now integrates **five stages**: YOLOv8 object detection for spatial localisation of ingredients, a switchable ingredient identifier (either Google Gemini Vision or a fine-tuned EfficientNet-B0 classifier), Gemini Flash for recipe generation with hard dietary constraints, and Groq's Llama3 model as an independent recipe verification layer.

A central contribution of this work is the creation of an original **Indian Ingredients Dataset** comprising 20 classes and approximately 10,000 images, covering culturally specific pantry staples including moong dal, haldi, paneer, hing, amchur, and saffron — for which no equivalent public benchmark exists. This dataset was used both to fine-tune EfficientNet-B0 and to evaluate both models alongside the existing Freiburg Groceries Dataset.

Evaluation was conducted across two datasets and two identification approaches. On the Indian Ingredients Dataset, Gemini Vision achieved **90.0% Top-1 accuracy** in a zero-shot setting, while the fine-tuned EfficientNet-B0 achieved **76.75% Top-1 (90.75% Top-3)** — reaching 85% of Gemini's performance with a local, offline model trained on 500 images per class. On the Freiburg Groceries Dataset, Gemini Vision achieved **81.6% Top-1 accuracy**. A key empirical finding is that fine-tuning EfficientNet on Indian ingredient data causes **catastrophic forgetting**: its Freiburg accuracy drops from 10.2% (random baseline) to 1.8%, quantifying the cost of domain specialisation. A Groq/Llama3 verification layer evaluates generated recipes on four dimensions: ingredient usage, serving accuracy, recipe coherence, and dietary preference adherence.

Introduction

The intersection of computer vision and natural language generation has opened new possibilities for intelligent food-related applications. Identifying available ingredients and suggesting appropriate recipes are tasks that are intuitive to humans but remain challenging for automated systems, particularly when ingredients appear in varied lighting conditions, at different stages of preparation, or in cluttered kitchen environments.

This project addresses these challenges through a hybrid pipeline that combines classical computer vision with modern large language models. A particular focus has been placed on the **Indian cooking context**, which presents unique challenges that existing systems

are not designed to handle. Indian kitchens contain 20 or more distinct spices and lentils, many of which are visually nearly identical — haldi, dhania, and red chilli powder are all fine coloured powders; moong dal and chana dal are both small yellow split lentils. No existing public dataset adequately covers these ingredients, and general-purpose vision models fail on them systematically.

The motivation for this extended work is practical: a user who photographs their kitchen should receive a recipe suggestion tailored to their actual pantry and cultural context, without typing a single ingredient. The system is designed to run on consumer hardware, using free-tier API services, and is deployable via a Streamlit web interface.

The key research question driving the evaluation is: how much of a large zero-shot vision model's performance can be recovered by a small fine-tuned local model? This comparison — between Gemini Vision (zero-shot, API-based) and EfficientNet-B0 (fine-tuned, local) — is the central technical contribution of the extended work.

Literature Review

Food computing has emerged as an interdisciplinary research area that applies AI, computer vision, and natural language processing to analyse food-related data including recipes, images, and nutritional information. The field is inherently multimodal because human perception of food involves visual appearance, textual descriptions, taste, and contextual knowledge. As a result, computational systems designed for food understanding often integrate multiple modalities — images, ingredient lists, cooking instructions, and nutritional metadata.

Recent advances in machine learning and large multimodal models have significantly accelerated research in this domain. A comprehensive survey on multimodal food learning highlights that modern food computing systems aim to relate information across different modalities to perform tasks such as food recognition, recipe retrieval, recipe generation, recommendation, and dietary analysis. The Food-101 dataset established a benchmark of 101 food categories, while the Freiburg Groceries Dataset specifically targets grocery and ingredient recognition in realistic settings. More recent work has explored fine-grained food segmentation with FoodSeg103.

Food recognition using convolutional neural networks (CNNs) formed the foundation of early food computing research. EfficientNet (Tan & Le, 2019) introduced a principled scaling method for CNNs that achieves state-of-the-art accuracy with significantly fewer parameters than prior architectures, making it well-suited to fine-tuning on small domain-specific datasets. Transfer learning — adapting a model pretrained on a large general dataset (ImageNet) to a specialised downstream task — has been shown to be highly effective in food recognition contexts.

Recipe generation from images or ingredient lists using large language models has gained substantial traction. LLaVA-Chef and ChefFusion demonstrate the potential of multimodal foundation models in creative culinary applications. Google's Gemini family of models provides state-of-the-art vision-language understanding, including zero-shot food and ingredient identification from photographs.

A notable gap in existing food computing literature is the lack of datasets covering Indian ingredients at the granularity of individual spices, lentils, and cooking staples. Work by Kumar et al. (2025) on indigenous food datasets highlights the systematic underrepresentation of non-Western culinary traditions in existing benchmarks, a gap this project directly addresses.

Problem Statement

Traditional recipe recommendation systems rely on users manually inputting a list of available ingredients, which is time-consuming and requires prior knowledge of ingredient names. Computer vision offers the potential to automate this step by identifying ingredients directly from images; however, two significant challenges exist.

First, ingredient identification from images is inherently difficult. Many ingredients look similar (e.g., atta and maida, salt and sugar), change appearance when processed (e.g., diced vs. whole onion), or are only identifiable through packaging (e.g., canned fish, bottled sauces). Standard object detection models trained on general datasets such as COCO 2017 cover only approximately 10–15 food-related classes out of 80 total. This limitation is especially acute for Indian cooking, where haldi, jeera, and dhania powder are visually nearly identical, and where moong dal, chana dal, and masoor dal differ only in subtle colour and size.

Second, recipe generation from a set of ingredients is a complex natural language task. A valid recipe must not only reference available ingredients but also produce coherent instructions, appropriate quantities, realistic timings, and respect user dietary constraints. Rule-based approaches are inadequate given the combinatorial complexity of culinary combinations. Furthermore, simply generating a plausible-looking recipe is insufficient — a robust system must be able to verify whether the generated recipe is coherent, correctly portioned, and genuinely compliant with stated dietary preferences.

Third, cultural context is absent from existing systems. Existing food computing pipelines are predominantly trained and evaluated on Western datasets. Indian cuisine involves ingredient combinations, spice profiles, and cooking techniques that differ fundamentally from Western cooking — and no existing benchmark dataset covers the Indian pantry staples that are central to daily Indian home cooking.

System Design and Architecture

The extended Rasoi AI system is structured as a five-stage sequential pipeline. Each stage produces a structured output that serves as input to the next stage. The key architectural extension from the initial three-stage design is the introduction of a switchable identifier in Stage 2 and an independent verification layer in Stage 5.

Pipeline Overview

1. YOLOv8 Object Detection: The input image is passed to YOLOv8n pretrained on COCO 2017. The model returns bounding boxes, class labels, and confidence

scores for all detected objects. Individual ingredient crops are extracted from bounding box coordinates and passed to Stage 2.

2. Ingredient Identification (Switchable): The user selects one of two identification approaches: (a) Gemini Vision, which receives the full image and returns a structured list of all identified ingredients with confidence and quantity estimates; or (b) fine-tuned EfficientNet-B0, which classifies each YOLO crop individually against 1020 classes (1000 ImageNet + 20 Indian ingredient classes). If no YOLO detections are found, the full image is classified as a single crop.
3. Recipe Generation: The identified ingredient list, combined with user-specified dietary preferences and serving count, is sent to Gemini Flash. Hard dietary constraints are enforced in the prompt rather than as soft suggestions. The model returns a complete recipe in structured JSON format.
4. Recipe Verification: The generated recipe is independently evaluated by Groq's Llama3 model on four dimensions: ingredient usage, serving accuracy, recipe coherence, and dietary preference adherence. The verifier is made aware of any detected ingredients correctly excluded due to dietary preferences, so exclusions are scored as compliant rather than penalised.

Rationale for Dual Identification Approaches

A natural question arises as to why both a fine-tuned local model and a zero-shot API model are offered, rather than selecting one. The dual approach reflects a fundamental tradeoff:

- **Gemini Vision** offers richer output (multiple ingredients simultaneously, quantity estimates, confidence levels) and works out-of-the-box on any ingredient including Indian spices, but requires internet connectivity, is rate-limited on the free tier, and is a black box.
- **Fine-tuned EfficientNet-B0** runs entirely offline with no API cost, is interpretable (produces softmax probabilities over a fixed vocabulary), and can be validated and audited. Its limitation is one prediction per YOLO crop and a vocabulary restricted to 1020 classes.

The switchable architecture allows users to choose based on their context: Gemini for general kitchen photos, EfficientNet for Indian ingredient identification in offline or cost-constrained settings.

Indian Ingredients Dataset

A central original contribution of this project is the creation of the **Indian Ingredients Dataset** — a purpose-built image classification dataset covering 20 classes of Indian pantry ingredients. No equivalent public dataset exists for these ingredients at this scale and granularity.

Motivation

Existing food computing datasets are predominantly Western-centric. The Freiburg Groceries Dataset covers 25 categories including pasta, chips, and soda, but contains zero Indian ingredient classes. Food-101 covers prepared dishes but not raw pantry ingredients. This gap means that any model evaluated only on Freiburg cannot be considered representative of performance in an Indian kitchen context.

Indian spices and lentils present unique computer vision challenges: haldi (turmeric), red chilli powder, and dhania (coriander) powder are all visually similar fine powders; moong dal and chana dal are split lentils with subtle colour differences; and hing (asafoetida) and amchur (dry mango powder) are visually nondescript. Even human identification can be unreliable without contextual cues such as smell or packaging.

Dataset Classes

The dataset comprises the following 20 classes:

Class Name	Hindi Name	Class Name	Hindi Name
Moong Dal	मूंग दाल	Cloves	लौंग (Laung)
Chana Dal	चना दाल	Besan	बेसन
Haldi	हल्दी (Turmeric)	Paneer	पनीर
Cardamom	इलायची (Elaichi)	Ghee	घी
Cumin	जीरा (Jeera)	Tamarind	इमली (Imli)
Coriander Seeds	धनिया (Dhania)	Curry Leaves	कड़ी पत्ता
Mustard Seeds	राई (Rai)	Methi	मेथी (Fenugreek)
Red Chilli Powder	मिर्च पाउडर	Hing	हींग (Asafoetida)
Garam Masala	गरम मसाला	Amchur	अमचूर (Dry Mango)
Cinnamon	दालचीनी (Dalchini)	Saffron	केसर (Kesar)

Collection and Statistics

- **Total images:** ~10,000 (approximately 500 per class)
- **Collection method:** Web-scraped and curated for this project
- **Image conditions:** Varied lighting, angles, and presentation styles
- **Data split:** Stratified 70% train 30% test using StratifiedShuffleSplit with random_state=42 for reproducibility
- **Training images:** ~7,000
- **Test images:** ~1,500

Stratified sampling ensures that each class is represented proportionally in each split, preventing imbalance caused by classes with slightly fewer than 500 images. The test set indices are saved in the model checkpoint to prevent data leakage during evaluation — the fine-tuned EfficientNet is evaluated only on images it never saw during training.

EfficientNet-B0 Fine-Tuning

Architecture: Class Expansion

EfficientNet-B0, pretrained on ImageNet's 1000 classes, was chosen as the base model for fine-tuning due to its strong accuracy-to-parameter ratio and fast CPU inference time (~100ms per image on a MacBook Air M1). Rather than replacing the classifier head entirely, we expanded it to support 1020 classes:

- **Rows 0–999:** Exact pretrained ImageNet weights copied from the original 1000-class classifier head. These neurons are preserved and updated only with a very small learning rate.
- **Rows 1000–1019:** 20 new neurons, one per Indian ingredient class, randomly initialised. These are trained at a higher learning rate.

This architecture allows the model to predict any ImageNet object (banana, cat, broccoli) OR any of the 20 Indian ingredient classes from a single forward pass, with softmax over all 1020 outputs. The highest-scoring class wins regardless of whether it is an ImageNet class or an Indian ingredient.

Training Setup

- **Dataset:** ~7,400 training images across 20 Indian ingredient classes
- **Hardware:** Google Colab T4 GPU
- **Training time:** Approximately 10 minutes for 15 epochs
- **Batch size:** 128
- **Optimiser:** AdamW with weight decay 1×10^{-4}
- **Backbone learning rate:** 1×10^{-5} (very low — preserve ImageNet features)
- **Classifier head learning rate:** 1×10^{-3} (higher — train new Indian neurons fast)
- **LR schedule:** Cosine annealing from initial LR to 1×10^{-6}
- **Loss:** Cross-entropy with label smoothing 0.1
- **Augmentation:** Random crop, horizontal flip, rotation (±15°), colour jitter (brightness, contrast, saturation, hue)

Random Baseline Model

To isolate the contribution of fine-tuning from the architecture itself, a random baseline model was also constructed. This model has identical architecture (1020-class head) but the 20 Indian neurons were never trained — they remain as random initialisation. The

pretrained ImageNet weights for neurons 0–999 are preserved exactly. This baseline allows us to measure: (a) what the model knows from ImageNet alone about Indian ingredients, and (b) whether adding random neurons without training them affects general performance. Comparing random vs. fine-tuned isolates the specific contribution of training on our Indian dataset.

Implementation

Project Structure

The project is organised as a Python directory with the following key files:

- **pipeline.py:** Core pipeline module containing YOLODetector, GeminiVisionIdentifier, EfficientNetClassifier, RecipeGenerator, and GroqRecipeValidator classes.
- **app.py:** Streamlit web application with model switching UI and integrated Groq verification display.
- **train_efficientnet.py:** EfficientNet-B0 fine-tuning script with stratified splits, differential learning rates, and checkpoint saving with test indices.
- **train_efficientnet_expanded.py:** Expanded 1020-class training script producing both the fine-tuned and random baseline checkpoints.
- **create_random_expanded.py:** Standalone script that creates the random baseline checkpoint from pretrained weights without training.
- **evaluate_efficientnet.py:** Evaluation script for both EfficientNet variants on both datasets, using batched inference.
- **evaluate_gemini.py:** Evaluation script for Gemini Vision on Freiburg and Indian datasets, with rate limit handling and auto-retry.
- **.env:** Environment file storing Gemini and Groq API keys.

Technical Details

YOLOv8 Integration

YOLOv8n weights are loaded via the Ultralytics Python library. The nano variant was selected for its minimal memory footprint and fast CPU inference. Bounding box coordinates are clamped to image bounds and each detection is cropped as a PIL Image object for downstream classification. When no detections are found, the full image is passed as a single crop.

Gemini Vision API

Images are passed to the Gemini API as PIL Image objects via the google-generativeai SDK. A system instruction establishes the model as a culinary vision AI and enforces structured JSON output. The user prompt specifies the exact schema including ingredient name, quantity estimate, confidence (high/medium/low), and descriptive notes. Responses are stripped of accidental markdown fencing before JSON parsing.

EfficientNet Classifier

At inference time, the EfficientNet classifier loads the fine-tuned checkpoint which contains the model state, idx_to_class mapping (all 1020 labels), and the saved test indices from training. YOLO crops are transformed with a deterministic eval transform (resize 256, centre crop 224, ImageNet normalisation) and passed one at a time. Softmax is applied over all 1020 logits and the highest-scoring class is returned with its confidence. Duplicate class predictions across multiple crops are deduplicated.

Recipe Generation — Dietary Constraints

The initial implementation used soft dietary preference suggestions in the recipe prompt. This was found to be unreliable — the LLM would sometimes generate recipes that technically acknowledged a restriction but included minor violations. The updated implementation enforces dietary constraints as hard rules with explicit definitions:

- Vegetarian: absolutely no meat, poultry, or seafood. Eggs and dairy are permitted.
- Vegan: absolutely no meat, poultry, seafood, dairy, eggs, or honey.
- Low-Carb: total carbohydrates under 20g per serving. No rice, bread, pasta, or sugar.

These rules are placed as a prominent MANDATORY section at the top of the prompt, before the ingredient list, to ensure high attention weight from the model.

Streamlit Application

The web interface provides a sidebar for configuring servings (1–6), dietary preferences (Vegetarian, Vegan, Low-Carb), and model selection. A radio button allows switching between Gemini Vision and EfficientNet. When EfficientNet is selected, a text input accepts the path to the fine-tuned checkpoint. The results area displays the original image alongside YOLO annotations, identified ingredient chips with confidence colour coding, the full recipe card, and the Groq verification scores.

Recipe Verification with Groq

Motivation

Large language models are proficient at generating text that appears coherent and well-structured. However, plausibility does not guarantee validity. A generated recipe may list correct ingredients but specify quantities incompatible with the requested serving count, include cooking steps in an illogical order, or nominally comply with a dietary restriction while subtly violating it. An automated verification layer provides an independent check on recipe quality without requiring human evaluation.

Architecture

Groq's inference API running Llama3-8b-instant was selected for the verification stage due to its fast inference speed (~1 second for structured responses), free-tier availability, and strong instruction-following capability. The verifier operates as a completely independent call from the recipe generator — it receives the generated recipe JSON, the

detected ingredient list, the requested serving count, and the stated dietary preferences, and produces a structured evaluation.

Evaluation Dimensions

The verifier scores the recipe on four dimensions, each rated 1–5 with a one-sentence justification:

- **Ingredient Usage (1–5):** Are the detected ingredients actually used as primary components of the dish, not merely mentioned as optional garnish?
- **Serving Accuracy (1–5):** Are ingredient quantities proportionate and realistic for the exact number of servings requested?
- **Recipe Coherence (1–5):** Are the cooking steps in a logical sequence? Do they collectively lead to a real, edible, complete dish? Steps out of order, missing steps, or instructions that would not produce the named dish are penalised.
- **Preference Adherence (1–5):** Does the recipe fully comply with the stated dietary preferences? Specific violations are listed. Correctly excluded ingredients (e.g., bacon detected but not used because preference is Vegetarian) are scored positively, not penalised.

An overall score (1–5) and one-sentence verdict are also returned. The verifier prompt instructs Llama3 to classify detected ingredients that were excluded from the recipe due to dietary constraints as evidence of correct behaviour, preventing the verifier from erroneously penalising compliant recipes for not using all detected ingredients.

Integration

The Groq verification call adds approximately 1–2 seconds to pipeline execution time. Results are displayed in the Streamlit interface below the recipe, with colour-coded bar indicators (green for scores ≥4, amber for 3, red for ≤2) and explicit violation lists when applicable. Verification results are also included in the downloadable JSON output.

Evaluation Methodology

Datasets

Freiburg Groceries Dataset

The Freiburg Groceries Dataset contains approximately 5,000 images across 25 grocery and ingredient categories captured in realistic supermarket and kitchen settings. For EfficientNet evaluation, 20 images per class were randomly sampled (500 total). For Gemini evaluation, 10 images per class were sampled (250 total) to remain within API rate limits. All sampling used random seed 42 for reproducibility.

Indian Ingredients Dataset (Ours)

20 images per class were sampled from the held-out test split for EfficientNet evaluation (400 total). This ensures strict data separation: images in the test split were never seen

during training or validation. For Gemini evaluation, 10 images per class were sampled (200 total).

Metrics

Top-1 Accuracy: The proportion of images for which the model's single highest-confidence prediction matches the ground truth class. For Gemini (which returns a ranked list), Top-1 checks only the first returned ingredient. For EfficientNet, Top-1 checks only the class with the highest softmax probability.

Top-3 Accuracy: The proportion of images for which the correct class appears anywhere in the model's three highest-confidence predictions. This metric captures whether the model "knows" the right answer even when not fully confident in it. For a recipe application where the user might confirm from a short list, Top-3 accuracy is arguably the more practically relevant metric.

Matching is performed via case-insensitive substring search against a manually curated synonym list for each class. For example, the Freiburg class PASTA accepts matches for 'pasta', 'spaghetti', 'noodle', 'penne', and 'fusilli'. For Indian ingredients, both English and Hindi synonyms are included (e.g., HALDI accepts 'haldi' and 'turmeric'; HING accepts 'hing', 'asafoetida', and 'asafetida').

Notes on Evaluation Methodology

The original report evaluated Gemini on Freiburg using 5 images per class with a limited synonym list, yielding 64.0% Top-1. The updated evaluation uses 10 images per class with a substantially expanded and corrected synonym list (including uppercase class name matching corrected to match the actual Freiburg folder naming convention). These methodology differences — not model changes — account for the difference between 64.0% and 81.6% in Gemini's Freiburg results.

For EfficientNet evaluation, the fine-tuned model is evaluated exclusively on the held-out test split (indices saved in the checkpoint) to prevent data leakage. The random baseline model, having never been trained on any data, is evaluated on the full dataset.

Results

EfficientNet-B0 Results

Table 1 presents Top-1 and Top-3 accuracy for both EfficientNet variants across both datasets.

Table 1: EfficientNet-B0 Evaluation Results

Model	Dataset	Images	Top-1 Acc	Top-3 Acc
Random Expanded	Indian	400	**0.0%**	0.5%
Fine-tuned Expanded	Indian	400	**76.75%**	**90.75%**
Random Expanded	Freiburg	500	10.2%	14.4%

Fine-tuned Expanded	Freiburg	500	**1.8%**	2.8%

Gemini Vision Results

Table 2 presents Gemini Vision results across both datasets in a zero-shot setting.

Table 2: Gemini Vision Evaluation Results (Zero-Shot)

Dataset	Images	Classes	Top-1 Acc	Top-3 Acc
Indian Ingredients (Ours)	200	20	**90.0%**	**91.0%**
Freiburg Groceries	250	25	**81.6%**	**85.2%**

Gemini Vision — Freiburg Per-Class Results

Table 3 presents Gemini's per-class Top-1 accuracy on the Freiburg dataset (10 images per class).

Table 3: Gemini Vision Per-Class Accuracy — Freiburg Groceries Dataset

Class	Top-1 Acc	Class	Top-1 Acc	Class	Top-1 Acc
BEANS	**100%**	JUICE	**80%**	TOMATO SAUCE	**90%**
CAKE	**10%**	MILK	**90%**	VINEGAR	**90%**
CANDY	**40%**	NUTS	**80%**	WATER	**100%**
CEREAL	**90%**	OIL	**100%**		
CHIPS	**70%**	PASTA	**100%**		
CHOCOLATE	**100%**	RICE	**100%**		
COFFEE	**100%**	SODA	**70%**		
CORN	**100%**	SPICES	**10%**		
FISH	**80%**	SUGAR	**80%**		
FLOUR	**100%**	TEA	**80%**		
HONEY	**100%**	JAM	**80%**		

Discussion

Fine-Tuning vs Zero-Shot Performance

The central research question was: how much of a large zero-shot model's performance can a small fine-tuned local model recover? The results provide a clear answer. Fine-tuned EfficientNet achieves 76.75% Top-1 and 90.75% Top-3 on Indian ingredients — reaching 85% of Gemini's 90.0% Top-1 accuracy using only 500 images per class of training data and approximately 10 minutes of GPU training. The 13-percentage-point gap between them represents the cost of local inference: no internet dependency, no API rate limits, and no cost per query, at the price of 13% accuracy on Top-1.

Top-3 accuracy of 90.75% for EfficientNet is particularly useful for the recipe application context. If the model's top-3 predictions are presented to the user for confirmation before recipe generation, the effective accuracy matches Gemini's performance. This suggests that a human-in-the-loop confirmation step after EfficientNet identification could substantially close the gap to Gemini.

Catastrophic Forgetting

The most scientifically significant finding is the behaviour of the fine-tuned model on the Freiburg dataset. The random expanded baseline — which has never been trained on any data — achieves 10.2% Top-1 on Freiburg purely from its preserved ImageNet backbone. After fine-tuning on Indian ingredients, the model's Freiburg performance drops to **1.8%** — substantially below the untrained baseline.

This is a textbook instance of catastrophic forgetting (also called catastrophic interference), a well-documented phenomenon in continual learning literature. When the model is exposed exclusively to Indian ingredient images during fine-tuning, the backbone's feature representations gradually shift toward Indian visual patterns — powders, spices, split lentils — even when the backbone learning rate is set very low (1×10^{-5}). After fine-tuning, when Freiburg images are presented, the backbone extracts "Indian-like" features for them, causing the wrong output neurons to activate.

This result is not a failure of the training procedure; it is an empirical demonstration of a fundamental tradeoff. Every percentage point gained on Indian ingredients costs generalisability on out-of-domain classes. This finding directly motivates the dual-model architecture: EfficientNet serves as an Indian-specialist offline model, while Gemini serves as the general-purpose online identifier. The architecture design is not arbitrary but is empirically grounded in this tradeoff.

Gemini Failure Analysis

Gemini's two lowest-scoring Freiburg classes — CAKE (10%) and SPICES (10%) — warrant examination. The Freiburg CAKE class contains images of packaged biscuits and confectionery items that do not visually resemble a traditional cake. Gemini correctly identifies these as 'biscuits' or 'pastries', but our substring matching requires the string 'cake' to appear in the prediction, causing a false negative. Similarly, Freiburg's SPICES class is a heterogeneous catch-all covering jars of mixed spices, ground powders, and whole seeds. Gemini identifies specific spices ('cumin', 'paprika', 'oregano') correctly — but the coarser ground truth label SPICES does not match.

These are evaluation methodology limitations rather than model failures. Gemini's predictions in these cases are semantically more accurate than the ground truth labels. A finer-grained evaluation methodology with soft semantic matching (e.g., embedding similarity between predicted and ground truth labels) would likely show higher effective accuracy for these classes.

Practical Implications

The results support a tiered deployment strategy: Gemini Vision for general kitchen photos where internet is available and API costs are acceptable; fine-tuned EfficientNet for Indian ingredient-specific scenarios, offline use, or applications where API dependency is undesirable. The Groq verification layer adds meaningful quality assurance to the recipe generation stage with minimal latency overhead, and the hard dietary constraint enforcement in the recipe prompt substantially reduces preference violations compared to the original soft-suggestion approach.

Limitations

- **YOLO trained on COCO — no Indian ingredient classes:** YOLOv8n recognises only approximately 10–15 food-related COCO classes. Indian ingredient images rarely contain COCO food objects, causing YOLO to find no detections and triggering full-image classification instead of crop-level classification. The spatial localisation benefit of YOLO is largely lost for Indian ingredient photos.
- **Catastrophic forgetting:** Fine-tuning EfficientNet on Indian data degrades Freiburg performance from 10.2% to 1.8%. A single model cannot be simultaneously expert in Indian ingredients and general Western groceries without mixed-dataset training or continual learning techniques such as Elastic Weight Consolidation (EWC).
- **EfficientNet: one prediction per crop:** EfficientNet is a single-label classifier. A photo of 5 Indian ingredients requires 5 separate YOLO bounding boxes to identify all 5. In practice, YOLO rarely produces enough accurate boxes for Indian ingredients, limiting the usefulness of crop-level EfficientNet classification for complex kitchen photos.
- **Label granularity gaps in Freiburg evaluation:** The Freiburg SPICES class is a heterogeneous catch-all. Gemini's specific predictions (cumin, paprika) are semantically correct but do not substring-match 'spices', causing artificially low accuracy. This is an evaluation limitation, not a model limitation.
- **Packaged goods:** Both YOLO and Gemini struggle with ingredients whose identity is encoded in packaging text (canned fish, bottled sauces). OCR integration could address this.
- **Groq recipe verification:** The Groq verifier is itself an LLM and inherits LLM limitations. Its scores are judgements, not ground truth. A controlled human evaluation comparing Groq scores to human ratings was not performed.

- **API dependency:** The system requires valid Gemini and Groq API keys and is dependent on these services remaining available. Model deprecations can break the pipeline without code changes.
- **Quantity accuracy:** Quantity estimates generated by Gemini (e.g., ‘approximately 2 medium’) are not formally evaluated as no ground truth quantity data exists in either dataset.
- **Cultural diversity of training data:** The scraped Indian ingredients dataset may contain images from non-Indian contexts (e.g., Indian spices in Western packaging or photography styles), introducing domain shift between training and real-world deployment conditions.

Future Work

- **Fine-tune YOLO on Indian ingredients:** The highest-impact single improvement. Training YOLOv8 on annotated Indian ingredient bounding box data would enable accurate spatial localisation of paneer, curry leaves, haldi, and other ingredients, making the crop-level classification pipeline effective for Indian kitchens. This would require an object detection dataset with bounding box annotations, distinct from the classification dataset used here.
- **Continual learning to prevent catastrophic forgetting:** Techniques such as Elastic Weight Consolidation (EWC), Learning Without Forgetting (LwF), or joint training on both Freiburg and Indian data simultaneously would allow a single EfficientNet model to be accurate on both domains. This eliminates the need for users to choose between a specialist and a generalist model.
- **Multi-label EfficientNet:** Retraining EfficientNet as a multi-label binary classifier (predicting presence/absence of each of the 20 Indian ingredients simultaneously) would allow it to identify multiple ingredients from a single image — matching Gemini’s core capability. This would require multi-label training data with verified co-occurrence annotations.
- **Expand the Indian ingredients dataset:** The current 20-class, ~500 images/class dataset is a starting point. Expanding to 100+ classes covering regional Indian ingredients (kokum, fenugreek varieties, regional dal varieties), multiple image conditions (natural light, blurry, packaged), and different preparation states (whole vs ground, raw vs cooked) would substantially improve model robustness.
- **Regional language recipe generation:** Generating recipes in Hindi, Tamil, Telugu, or Bengali would significantly increase accessibility. The Gemini API supports multilingual generation and the recipe prompt can be extended to include a target language parameter.
- **Human evaluation of Groq verification:** A controlled study comparing Groq’s recipe scores to human expert ratings would establish the reliability of LLM-as-judge for culinary evaluation. This would strengthen the scientific validity of the verification layer.

- **Mobile application with camera integration:** A mobile app would allow real-time camera pointing at kitchen ingredients rather than static image uploads. EfficientNet's fast inference time (~100ms on modern mobile hardware) makes real-time classification feasible for the offline, Indian-specialist use case.

Conclusion

This project demonstrates a complete, working end-to-end pipeline for automated ingredient identification and recipe generation from kitchen photographs, with a specific focus on the Indian cooking context. The system integrates YOLOv8 for object localisation, a switchable identifier (Gemini Vision or fine-tuned EfficientNet-B0), Gemini Flash for recipe generation with hard dietary constraints, and Groq's Llama3 for independent verification.

Five findings are worth highlighting. First, the pipeline works end-to-end: a user can upload a kitchen photo and receive a structured, verified recipe without typing a single ingredient. Second, fine-tuning a small local model on 500 images per class recovers 85% of the performance of a trillion-parameter zero-shot model on the same task, achieving 76.75% Top-1 on Indian ingredients vs Gemini's 90.0%. Third, this specialisation comes at a measurable cost: fine-tuning on Indian data causes the model's Freiburg accuracy to drop from 10.2% to 1.8%, demonstrating catastrophic forgetting empirically. Fourth, the original Indian Ingredients Dataset — 20 classes, ~10,000 images — fills a genuine gap in existing food computing benchmarks and can serve as an independent resource for future research. Fifth, the Groq verification layer demonstrates that two-LLM cross-checking — one generating, one evaluating — is a practical pattern for improving the trustworthiness of LLM-generated culinary outputs.

Together, these contributions advance the state of automated recipe generation for culturally specific culinary contexts and provide a template for extending food computing systems beyond their current Western-centric scope.

References

Min, W., Jiang, S., Liu, L., Rui, Y., & Jain, R. (2025). Multimodal food computing: A survey of tasks, datasets, and methods. ACM Computing Surveys.

Salvador, A., Hynes, N., Aytar, Y., Marin, J., Ofli, F., Weber, I., & Torralba, A. (2017). Learning cross-modal embeddings for cooking recipes and food images. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. In Proceedings of the 36th International Conference on Machine Learning (ICML).

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., et al. (2017). Overcoming catastrophic forgetting in neural networks. Proceedings of the National Academy of Sciences, 114(13), 3521–3526.

Zhang, X., et al. (2025). SFOOD: A multimodal benchmark for food understanding and nutrition analysis. arXiv preprint arXiv:2507.04412.

Kumar, A., et al. (2025). What's Not on the Plate? A multimodal dataset for indigenous recipes and cultural AI. arXiv preprint arXiv:2509.16286.

Zhang, H., et al. (2025). RecipeGen: A multimodal dataset for step-by-step cooking instruction generation. arXiv preprint arXiv:2503.05228.

Jund, P., Abdo, N., Eitel, A., & Burgard, W. (2016). The Freiburg Groceries Dataset. arXiv:1611.05799.

Zhang, Y., et al. (2025). Designing systems capable of finding ingredients from food images using convolutional neural networks. Scientific Reports.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. CVPR 2016.

Zhang, Z., et al. (2024). LLaVA-Chef: Adapting multimodal large language models for recipe generation. arXiv preprint arXiv:2408.16889.

Li, Y., et al. (2024). ChefFusion: A multimodal foundation model for food understanding and recipe generation. arXiv preprint arXiv:2409.12010.

Google DeepMind. (2024). Gemini: A Family of Highly Capable Multimodal Models. arXiv:2312.11805.

Bossard, L., Guillaumin, M., & Van Gool, L. (2014). Food-101 – Mining Discriminative Components with Random Forests. ECCV 2014.

Anthropic. (2024). Claude [Large language model].

GroqCloud. (2024). Groq API documentation. https://console.groq.com/docs.